Rochas.AI.Manager — Arquitetura v2.2

Data: 2026-10-20 (rev. 2.1 em 2026-10-25; rev. 2.2 em 2026-10-29)
Status: vigente — a rev. 2.2 absorve a evolução de indução do `OpenCodeBridge` (2026-10-26 → 2026-10-29, 127 commits), fixa a matriz de paridade `OCB Core → AI.Manager.Bridge` e decide que **toda dinâmica corretiva de harness/tails/enunciados entra como dado no fluxo do AI.Manager** (§9.7). Substitui a organização do v1; o v1.2 permanece como histórico de intenção e as-built na seção 32.
Escopo: arquitetura da plataforma `OpenCodeBridge/AI.Manager` — domínio, aplicação, persistência, harness e indução, `AI.Manager.Bridge`/proxy, TestBench, workflow de aprovação, treinamento, relatórios, segurança, testes e roadmap.
Documentos relacionados: `Rochas.AI.Manager-Arquitetura v1.md` (§32 as-built), `REPORT-PENDINGS.md` (catálogo de relatórios), `BRIDGE-TRANSPORT.md` (transporte), `GUIA-INICIANTE.md` (rev. 2.2), `F1-F2-STATUS.md`, `ROADMAP-FIM-MIGRACAO-Q4.md`, `GUIA-TUNING-EQUIPE-TECNICA.md`.
PDF: `docs/ai-manager/pdf/Rochas.AI.Manager-Arquitetura v2.pdf` (gerado a partir deste `.md`).

---

1. Sumário executivo

O Rochas.AI.Manager saiu do papel e hoje é uma aplicação .NET 10 com treze projetos de produção (sete do núcleo, incluindo o SDK guarda-chuva `Core`, e seis do `AI.Manager.Bridge`) e duas suítes xUnit, cobrindo: catálogo de modelos e datasets, perfis de instrução e padrões, experimentos, jobs de treino com métricas, harness versionado (perfis, versões, anexos, políticas de tool, bindings, runs, capturas), workflow de aprovação com trilha de transições, API REST com Swagger, proxy OpenAI-compatible com saneamento de reasoning e coerção de tool calls, UI MVC com 18 controllers e 22 views, e exportação de relatórios em CSV/XLSX/PDF/HTML.

O porte de 2026-10-25 extraiu do OpenCodeBridge o núcleo protocol-only (`Responses↔chat`, harness, anti-repeat, breaker), os contratos, o client HTTP/SSE, o hosting de flags/DI e dois hosts (`Bridge.Api` REST+Swagger e `Bridge.Runner` CLI). O proxy legado do AI.Manager continua no ar até o cutover; o OpenCodeBridge fica como doador congelado.

A v2 consolida o que existe e corrige três distorções entre intenção e código:

1. **Execução de treino e motor numérico.** O job roda dentro da requisição HTTP e o "backend TorchSharp" é, na prática, um MLP em `double[]`; o modo avançado chama `python3`. A v2 trata isso como baseline explícito e condiciona o motor real a uma prova verificável.
2. **Relatórios.** O contrato promete recorte temporal e facetas; a implementação aplica apenas `Status` e pagina em memória. A v2 define o subsistema de relatórios com registry declarativo, SQL whitelisted e semântica de intervalo fechado/aberto.
3. **Chat e dashboard.** A view de chat (1.498 linhas) chama sete rotas que não existem no AI.Manager, e não há dashboard operacional. A v2 decide: portar os endpoints ou retirar a view; e transforma o `/health` em base do painel.

A regra editorial permanece a do v1 (§28.1/§28.14): capacidade só é "concluída" com código, teste e evidência.

A **rev. 2.2** acrescenta o eixo que faltava: **indução**. Entre 2026-10-26 e 2026-10-29 o `OpenCodeBridge` recebeu 127 commits quase todos sobre "por que o modelo pequeno não obedece" — dica que ensina forma inválida, breaker cego a erro de compilador, path relativo confundido com `cd`, args truncados, enunciado que dita mecânica. A v2.2 registra esse panorama (§9.6), fixa a matriz de paridade entre o Core do OCB e o `AI.Manager.Bridge` (§10.5) e decide que **os textos e enunciados de indução são dados versionados** (§9.7), semeados a partir do doador com hash de procedência. Estado medido em 2026-10-29: build 0 erros / 113 avisos; `Bridge.Tests` 144 PASS; `AI.Manager.Tests` 105 PASS + 1 SKIP; AI.Manager em `537fec1`, OCB em `3d8bd58`.

---

2. O que mudou da v1 para a v2

Tema	v1 (intenção)	v2 (baseline + alvo)
Estrutura	7 projetos incluindo `TrainingWorker`	13 projetos reais (7 núcleo + 6 Bridge) + 2 suítes; worker vira hosted service e depois projeto dedicado (ADR-10)
UoW/Repositórios	`IRepository` / `IUnitOfWork` no Domain	Contratos do pacote `Rochas.DapperRepository` como baseline; contratos próprios removidos ou implementados (ADR-11)
Treinamento	TorchSharp multi-modelo desde o início	MLP é baseline rotulado; TorchSharp só após prova (ADR-06); Python vira adaptador externo opcional (ADR-07)
Harness	Seção 31 como requisito	Modelado e semeado; `MainHarness` base + `HarnessTails` no Bridge, gate por cliente; falta Test Bench e vínculo de evidência (ADR-03/ADR-14)
Indução (novo na 2.2)	Implícita ("ajustar prompt")	Corpus de mecanismos medidos no OCB (dica/breaker/path/salvage/FsMini) + seed como dado versionado com hash de procedência (ADR-15…ADR-21)
Relatórios	"Estatísticas no admin"	Subsistema próprio com catálogo R1–R30, facetas e De/Até (ADR-04, ADR-05)
Bridge	Preservar contrato	`AI.Manager.Bridge` extraído: `/v1/responses`, `/v1/chat/completions`, hosts Api/Runner, harness/tails, anti-repeat/breaker; integração in-process no AI.Manager ainda pendente (ADR-13)
Chat	Reaproveitar chat web	View portada sem backend; decisão de portar endpoints (ADR-08)
Segurança	Autenticação/autorização genéricas	Chave JWT, seed e proxy precisam de hardening imediato (§15)
Documentação	Guideline extenso	Documento único de arquitetura + as-built no v1 + pendências separadas

3. Escopo, não-escopo e premissas

No escopo da v2

- Consolidar o as-built e impedir regressão silenciosa.
- Relatórios com facetas de FK e recorte temporal confiável.
- Ciclo de vida de job de treino fora da requisição HTTP.
- Evidência de harness ponta a ponta (perfil → versão → run → captura).
- Segurança mínima de produção e observabilidade operacional.
- Contratos de teste para bridge e rotas.

Fora do escopo (nesta versão)

- Treino real de Gemma 4/Llama/Qwen/DeepSeek com pesos completos.
- Multimodal, áudio, ChatTTS e app mobile (permanecem como horizonte).
- Migração para PostgreSQL.
- Substituição da bridge por outro protocolo.

Premissas

- Uma máquina de desenvolvimento com duas bridges locais (`:4124` GPU, `:4125` CPU) e servidores llama em `:4100` / `:4110` .
- `:4124` (Q4) é o **gate**; `:4125` (Q3) é **espelho pós-green**, nunca critério de aceite.
- O `Rochas.OpenCodeBridge` é **doador transitório**: recebe features até o cutover e depois é arquivado.
- SQLite é suficiente para o volume atual (centenas a milhares de registros por tabela).
- O usuário final é técnico; "modo básico" continua sendo o caminho recomendado.

4. Baseline tecnológico (fixado)

Item	Versão	Observação
.NET	10.0	`net10.0` em todos os projetos
ASP.NET Core	10.0.13	MVC + API + JwtBearer + OpenApi
Swashbuckle	10.3.1	Swagger em Development
Dapper / ORM	`Rochas.DapperRepository` 2.0.5	`GenericRepository<T>` , `IGenericRepository` , `IPersistenceRepository`
Especificação	`Rochas.Data.Specification` 1.6.3	usado como filtro de consulta
SQLite	`Microsoft.Data.Sqlite` 10.0.12	WAL, `Cache=Shared` , `busy_timeout=10000`
PDF/Excel	`Rochas.PDFGenerator` 1.3.0 / `Rochas.ExcelToJsonParser` 1.6.4	exportação de relatórios
Telemetria/log	`Serilog` 4.4.0 + `Serilog.AspNetCore` 10.0.0 + `Rochas.Telemetry` 1.1.1	console estruturado
Numérico	`TorchSharp` 0.107.0	referenciado, **não utilizado** no backend atual
Testes	xunit 2.9.3, Test.Sdk 17.14.1, coverlet 6.0.4	61 casos

5. Estrutura da solução

```
AI.Manager/
  Rochas.AI.Manager.slnx
  src/
    Rochas.AI.Manager.Domain/            # entidades, base, contratos, helpers
    Rochas.AI.Manager.Application/       # serviços, tools handlers, helpers de resposta
    Rochas.AI.Manager.Infrastructure/    # AppDb, ReportService, PasswordHasher, Seeding (seeders + assets), TestBench
    Rochas.AI.Manager.TrainingEngine/    # backend numérico + runner de processo + engine
    Rochas.AI.Manager.Core/              # SDK guarda-chuva (Domain+Application+Infrastructure+TrainingEngine)
    Rochas.AI.Manager.Api/               # REST, Swagger, proxy /v1 legado, tools
    Rochas.AI.Manager.Web/               # MVC, views, /health (sem seeder: só chama Infrastructure no boot)
    Rochas.AI.Manager.Bridge/            # núcleo protocol-only: Responses↔chat, harness, anti-repeat, breaker
    Rochas.AI.Manager.Bridge.Contracts/  # DTOs/manifesto/opções (sem ASP.NET)
    Rochas.AI.Manager.Bridge.Client/     # HTTP/SSE tipado
    Rochas.AI.Manager.Bridge.Hosting/    # flags/env, DI, bootstrap, BridgeComponent
    Rochas.AI.Manager.Bridge.Api/        # host REST+Swagger (/v1, /api/status, /api/contract)
    Rochas.AI.Manager.Bridge.Runner/     # host CLI serve/manifest (HttpListener)
  tests/
    Rochas.AI.Manager.Tests/             # 105 PASS + 1 SKIP (2026-10-29)
    Rochas.AI.Manager.Bridge.Tests/      # 144 PASS (2026-10-29)
  Data/Seeds/                            # baselines versionados (war-baseline.json)
  docs/ai-manager/                       # arquitetura v1/v2, guias, roadmap, pendências
  docs/ai-manager/pdf/                   # PDFs gerados a partir dos .md (v2 e guia)
```

Organização de seeders e execução (rev. 2.2, decisão ADR-18):

```
src/Rochas.AI.Manager.Infrastructure/
  Seeding/                    # TODOS os seeders (única camada que conhece o banco)
    HarnessBaselineSeeder.cs  # núcleo/tails/FsMini/notices/help desde harness-baseline.json
    ToolHelpSeeder.cs
    HintCatalogSeeder.cs
    NoticeCatalogSeeder.cs
    WorkflowSeeder.cs
    HarnessSeeder.cs
    BridgeFeatureSeeder.cs
    Assets/                   # harness-*.md, harness-tools.json, hint catalog (fonte do seed)
  TestBench/                  # TestBenchSeeder, WarBaselineReader
  Data/AppDb.cs               # schema
Data/Seeds/                   # baselines versionados (war-baseline.json, harness-baseline.json)
```

- **Web/Api não semeiam**: no boot chamam `Infrastructure` (`ISeederRunner`/`SeedAsync`) e seguem. View/controller nunca escreve catálogo de harness.
- **Runner de cenário é console** (`Bridge.Runner`): executa manifesto, aplica assertions e grava `BridgeResult`; não referencia `Infrastructure`, não abre SQLite. O AI.Manager persiste a evidência depois.
- **Exceção transitória:** os 5 seeders que hoje moram em `Web/Data` são dívida declarada (§6, violações) e migram para `Infrastructure/Seeding` na Fase 0.5.

Alvo incremental (não criar antes de precisar):

```
src/Rochas.AI.Manager.Worker/           # hosted service de fila (Fase 2)
src/Rochas.AI.Manager.Reporting/        # registry + executor SQL (Fase 1, ou pasta em Application)
src/Rochas.AI.Manager.Inference/        # clientes Ollama/llama-server (Fase 5)
```

---

6. Camadas e regras de dependência

Regra de ouro: `Domain` não conhece nada; `Application` conhece `Domain`; `Infrastructure` conhece os dois; `Web`/`Api` conhecem `Application`, `Infrastructure` e `TrainingEngine`; `TrainingEngine` conhece `Application` (para relatar progresso), nunca `Web`/`Api`.

Regra do Bridge: `Rochas.AI.Manager.Bridge` é o núcleo protocol-only e referencia apenas `Bridge.Contracts`; não referencia ASP.NET, CLI, Swagger, `Domain` ou `Application`. `Bridge.Client` referencia `Contracts`; `Bridge.Hosting` referencia `Contracts`; `Bridge.Api`/`Runner` referenciam `Bridge`, `Contracts` e `Hosting`. O sentido proibido é `Bridge` → `Api`/`Runner`/`Client`/`Web`.

Camada	Contém	Proibido
Domain	Entidades, base classes, enums, `ReportQuery/Result`, `EntityHashCode`	Dapper, SQLite, ASP.NET, TorchSharp
Application	Serviços (regras), tools handlers, `ReasoningSanitizer`, `ToolCallCoercer`	SQL direto, acesso a `HttpContext`, `Process` fora de `ProcessRunner`
Infrastructure	`AppDb` (schema), `ReportService`, hasher, **seeders + assets de seed** (`Seeding/`), TestBench (reader/seeder)	Regra de negócio, decisão de fluxo
TrainingEngine	`TrainingEngine`, backend, `TrainingRunner`	Controllers, views, banco
Bridge (core)	`ResponsesToChat`, `ChatToResponses`, `BridgeProxy`, `MainHarness`, `HarnessTails`, anti-repeat, breaker, `QwenStandard`, `ContextGuard`	ASP.NET, CLI, Swagger, `Domain`/`Application`, I/O de banco
Bridge hosts	`Api` (REST+Swagger), `Runner` (serve/manifest/run, **console**), `Hosting` (flags/DI), `Client`, `Contracts`	Regra de negócio do AI.Manager
Web / Api	Controllers finos, views, DI, `/health`; no boot apenas chamam o seeder da Infrastructure	Regra de negócio duplicada, SQL direto, **seeder próprio/tabela de harness**

Violações atuais (corrigir na Fase 0/1):

- `Application` referencia `Microsoft.Data.Sqlite` e `Rochas.DapperRepository` (deveria ser só Infrastructure).
- `ReportService` (Infrastructure) monta dicionários por reflexão e pagina em memória — aceitável como ponte, não como alvo.
- `ApprovalsController` consulta repositórios diretamente em vez de serviços.
- Contratos `Domain/Common/IRepository` e `IUnitOfWork` não são resolvidos no DI (código morto).
- **Cinco seeders moram em `Web/Data`** (`HarnessSeeder`, `HarnessScenarioSeeder`, `ScenarioTemplateSeeder`, `BridgeFeatureSeeder`, `WorkflowSeeder`) com assets `harness-*.md`/`harness-tools.json` no mesmo diretório; `TestBenchSeeder` já está no lugar certo (`Infrastructure/TestBench`). Migração para `Infrastructure/Seeding` é pré-requisito da §9.7 (ADR-18).

---

7. Modelo de domínio

7.1 Base

- `BaseEntity`: `Id`, `ParentId`, `Code`, `Name`, `IsActive`, `CreatedAt` (UTC).
- `AuditableEntity`: + `OwnerLogin`, `UpdatedAt`, `UpdaterId`, `LastUpdate`.

- `MultiEntityReferrer` : + `EntityHashCode` (djb2 do nome completo do tipo).
- `WorkFlowBasedEntity` : + `StatusId` , `NextStatusId` , `RequesterId` , `ApproverId` , `RequestNote` , `ApprovalNote` , `Approved` , com `SetStatus/RequestApproval/Approve/Reject` .

7.2 Agregados e invariantes

Agregado	Raiz	Invariantes
Catálogo de modelos	`ModelArtifact`	`status` ∈ {Draft, Approved, Deprecated}; aprovar só de Draft; descontinuar só de Approved
Dataset	`Dataset`	`format` obrigatório; `is_validated` só após validação; hash para integridade
Perfil de instrução	`InstructionProfile`	versionamento textual; aprovar só de Draft
Padrão de treino	`TrainingPattern`	`profile_id` e `model_artifact_id` opcionais, mas combinação deve existir quando ambos informados
Experimento	`Experiment`	Planned→Running→Completed→{Approved, Rejected}; rejeição exige motivo
Job de treino	`TrainingJob`	Queued→Running→{Completed, Failed, Cancelled}; `completed_at` só no fim; `best_accuracy` monotônica
Harness	`HarnessProfile` → `HarnessProfileVersion` → `HarnessAttachment`	versão imutável após aprovação; checksum obrigatório para aprovar; anexos por role
Política de tool	`ToolPolicy`	`allow` / `deny` disjuntos por role; `max_turns` / `stall_limit` > 0
Binding	`ModelBinding`	provider+model_id únicos; contexto ≥ max_tokens
Run de harness	`HarnessRun`	`profile_version_id` , `model_binding_id` e `evidence_id` obrigatórios no alvo
Card de aprovação	`TrainingWorkflowCard`	transições conforme `workflow_steps` ; aprovação/rejeição registram autor e nota
Auditoria	`WorkFlowTransition`	imutável; registra de/para, ação, autor e nota

7.3 Grafo de FKs lógicas (todas sem constraint física)

```
users ──requester_id/approver_id/updater_id──> training_workflow_cards
experiments.base_artifact_id | tuned_artifact_id ──> model_artifacts
training_jobs.model_artifact_id ──> model_artifacts
training_jobs.dataset_id        ──> datasets
training_jobs.experiment_id     ──> experiments
training_patterns.profile_id    ──> instruction_profiles
training_patterns.model_artifact_id ──> model_artifacts
harness_profile_versions.profile_id ──> harness_profiles
harness_attachments.version_id ──> harness_profile_versions
harness_runs.profile_version_id ──> harness_profile_versions
harness_runs.model_binding_id   ──> model_bindings
harness_runs.evidence_id        ──> bridge_captures (convenção, sem FK)
training_workflow_cards.experiment_id ──> experiments
training_workflow_cards.status_id/next_status_id ──> entity_statuses
workflow_steps.status_id/next_status_id ──> entity_statuses
workflow_transitions.(entity_hash_code, entity_id) ──> qualquer entidade
workflow_transitions.approver_id ──> users
* parent_id ──> self
```

7.4 Máquinas de estado

- **Artefato/Perfil/Harness:** DRAFT → APPROVED → DEPRECATED.
- **Experimento:** PLANNED → RUNNING → COMPLETED → APPROVED | REJECTED.
- **Job:** QUEUED → RUNNING → COMPLETED | FAILED | CANCELLED (cancelamento a partir de QUEUED/RUNNING).
- **Card:** NOVO → ACEITE | REPROVADO (com passos intermediários declarados em `workflow_steps`).

---

8. Persistência e dados

8.1 Baseline

- SQLite em `AppContext.BaseDirectory/ai.manager.db` , WAL, `synchronous=NORMAL` , `busy_timeout=10s` .
- Schema criado por `AppDb.Init()` com `CREATE TABLE IF NOT EXISTS` (19 tabelas) e migração ad-hoc via `ALTER TABLE ... try/catch` .
- Acesso por `GenericRepository<T>` do pacote, registrado manualmente no DI (≈20 registros por host).
- Datas gravadas como `TEXT` (ISO-like, UTC), o que exige normalização em qualquer filtro temporal.

8.2 Política de schema (v2)

1. Introduzir `schema_version` com scripts SQL versionados (`001_init.sql` , `002_...`), substituindo `ALTER TABLE` em `try/catch` .

2. Manter FKs **lógicas** no SQLite (ADR-02) e cobrir integridade com relatórios de órfãos (R29 do catálogo).
3. Índices mínimos por eixo de relatório: `created_at` em `training_jobs`, `harness_runs`, `bridge_captures`, `workflow_transitions`; `(status)` em `training_jobs`/`experiments`; `(profile_version_id)` e `(model_binding_id)` em `harness_runs`; `(entity_hash_code, entity_id)` em `workflow_transitions`.
4. `AppDb.Init` continua idempotente; scripts rodam antes do seed.
5. Toda escrita valida existência das FKs referenciadas (serviço), registrando erro claro em vez de aceitar órfão.

8.3 Transações

Operações que tocam duas tabelas (ex.: `Move` do card + `WorkFlowTransition`; `RunAsync` + `HarnessRun`) devem usar transação explícita via conexão compartilhada. Baseline atual não garante atomicidade; a v2 exige helper `IUnitOfWork` real ou `SqliteTransaction` no serviço de infraestrutura.

---

9. Harness como produto

9.1 Modelo (implementado)

`HarnessProfile` → `HarnessProfileVersion` (`mode`, `checksum`, `approved_by/at`) → `HarnessAttachment` (`kind`, `role`, `content`, `char_count`), com `ToolPolicy` por role e `ModelBinding` fora do texto do prompt.

Modos: `ClientNative` (default; cliente traz o harness), `BridgeBypass`, `BridgeLegacy`, `Hybrid`.

9.2 Regra de ouro

Harness descreve comportamento; infraestrutura fica na configuração. Modelo, provider, porta, contexto, temperatura, caminho local e limites numéricos não entram no texto. `HarnessLinter` já bloqueia 6 padrões (porta, `NNk`, temperatura, caminho, nome de modelo/provider, número grande).

9.3 Ciclo de vida

1. Criar perfil (Draft).
2. Adicionar versão com `mode` e `checksum`.
3. Anexar núcleo/tails por role (contador de caracteres).
4. Lint obrigatório; hardcode bloqueia aprovação.
5. Aprovar versão (`approved_by`, `approved_at`).
6. Registrar runs contra a versão + binding.
7. Comparar versões (diff de texto e tamanho) e promover/reverter.

9.4 Pendências para fechar a seção 31 do v1

- Test Bench (enunciado canônico por faixa, evidência de arquivos/rc/tokens/s/duração).
- Diff visual entre versões no admin.
- Matriz de capacidade por modelo (tool calling, thinking, contexto, taxa de sucesso).
- Geração de `agents/*.md` e `opencode.json` a partir do perfil.
- `HarnessRun` com perfil/versão/captura reais (hoje o proxy grava `0`).

9.5 MainHarness e tails no Bridge (porte 2026-10-25)

O `Rochas.AI.Manager.Bridge` separa **base** e **cauda**:

- `MainHarness.Apply(messages, enabled)` substitui o system do cliente pela base enxuta e preserva `<env>`, `<available_skills>`, `Today's date:` e instruções pequenas em `INSTRUCOES DO CLIENTE`. Não substitui o master do opencode (`# Code Mode`, `Instructions from:`, `You are opencode`, system > 6000 chars).
- O gate é o **cliente**, nunca o sufixo do modelo: o host clona `BridgeOptions` com `ForClient(User-Agent)`; a base só aplica quando `MainHarness && !OpencodeClient`. O WAR (`<modelo>-build`) recebe a base; o CLI do opencode preserva o Code Mode.
- Base e tails **coexistem**. `HarnessTails.DefaultBehavior` é o delta do canal (formato, tools primeiro, persistência, prioridade); escrita sem loop, caminhos e "2 tentativas" moram na base.
- `BridgeOptions.TailScope` = `managed` (default; todos menos o CLI do opencode), `all` (também no opencode) ou `off` (nenhuma).
- Flags: `--main-harness` (default no host), `--no-main-harness`, `--tail-scope managed|all|off`; ambiente `BRIDGE_MAIN_HARNESS`, `BRIDGE_TAIL_SCOPE`.

9.6 Panorama da indução — o que o OCB corrigiu entre 2026-10-26 e 2026-10-29

O intervalo entre a rev. 2.1 e esta rev. 2.2 registra **127 commits** no doador, praticamente todos sobre a mesma pergunta: *por que o modelo de 4B não obedece ao harness?* A resposta medida foi consistente — **o defeito estava na indução, não no modelo**: dica que exibe literal inválido, enunciado que dita a mecânica, breaker que não enxerga falha de tool, path relativo confundido com `cd`, args truncados pelo próprio anti-repeat.

9.6.1 Classes de defeito e o mecanismo que as fecha

Classe	Sintoma medido	Mecanismo (OCB)	Prova	Estado no `AI.Manager.Bridge`
Dica inválida	modelo copia `[for i in 1 .. n-1 do expr]` (forma que não compila)	regra "a dica ensina a forma CERTA e nunca exibe a errada"; `FsMini` descreve por sintoma, não por literal errado	`U-tails-path-simulation-rules` (exige a forma certa **e** ausência da errada), `fs-repair-probe.fsx`	ausente (sem `FsMini`)
Reparo cego	17 reparos idênticos em 30 turnos	`FsSyntaxRepair.Suggest` despacha pelo **primeiro** código FS, mede na fonte e, na 3ª repetição, manda reescrever com `write`	`U-fs-repair-*` (comprehension, lambda `\`, `ref`, `: nome:` no retorno, `else let`, aridade, uso antes da declaração)	ausente
Breaker cego a tool	loop `edit` + `dotnet build` sem nudge nenhum	`ToolsAntiRepeat.FailureMarkers` cobre `exit [1-9]`, `error fs/cs/msb/fsi\d+`, traceback, `undefined reference`; streak por **alvo mais frequente na janela** (sucesso intercalado não zera); nudge aceita 2 alvos	`U-breaker-compiler-loop`, `U-breaker-interleaved-loop`, `U-breaker-failure-markers`, `U-breaker-ignores-distinct-targets`	parcial (`FailureMarkers` ainda na forma `exited with code [1-9]`, sem os códigos FS/CS)
Path relativo	`edit` com `random_file.txt` após `cd temp_dir`	`WorkspaceGuard.Locate` (BFS depth 4, 3 achados) devolve "existe em: `<rel>` — use esse path"	`U-tools-path-hint`	ausente
Args truncados	`window=512` cortou `edit` no meio (`salvage-args-rejected`)	janela **própria** dos args (1024, `--anti-repeat-args-window`) com buffers separados no stream	`U-antirepeat-args-window`, `verify-antirepeat-args-window.py`	ausente (janela única)
Aviso pobre	13 turnos de `JSON invalido/truncado` sem exemplo	`ToolCallRepair` = cabeçalho + 3 exemplos válidos (parseados no pin) + `ToolHelp.For` (fonte única do schema)	`U-toolcall-salvage-notice`	ausente
Custo do salvage	nudge de salvage debitava turno do orçamento	`ToolCallRepair.IsSalvageNotice` fora do orçamento	`U-toolcall-salvage-notice`	ausente
Enunciado-receita	teste media obediência (ditava `MapGet`, `useEffect`, DI exata)	enunciado = problema + contrato observável; mecânica no harness/tool-help; assert estrutural quando o nome é ambíguo	`E-ent-*` (5 PASS/1 FAIL → estabilizado)	a replicar no seed (§9.7)
Plano em vez de código	"Plano de Implementação … aguardando confirmação" em pedido textual	`DefaultBehaviorNoTools` define a frase de fechamento como **entregável final**, com proibição explícita de "plano de arquitetura" e de elisão (`// ...`)	`E-ent-crud-product`, `E-ent-dashboard`, `E-ent-redis`	ausente (só uma cauda `Behavior`)
Tools como texto	request sem `tools` devolvia `{"tools":[{"type":"shell"...` truncado	`Compose(..., hasTools:false)` troca a cauda e remove "TOOLS PRIMEIRO"	`U-main-harness-tools-tail`	ausente (`Compose` não tem `hasTools`)
Orçamento irreal	`E-ent-redis` falhava com 14 passos e passava com 24	teto **medido** (24) e `calls=` na mensagem de diagnóstico	`E-ent-redis` PASS 178,3 s (`calls=19`)	a replicar no loop JSON
Diagnóstico por restart	contador acumulado e painel com enunciado antigo	`POST /api/diag/reset?probe=1&files=1&panel=1`; reset **uma vez por bateria**, `step_start` não zera	`U-diag-reset`, `U-diag-reset-panel`	ausente
Painel mudo	step só aparecia depois de pronto	`step_start` publicado como `RUN`	`U-circuit-publish-run`	ausente (sem driver no Bridge)
Fail-fast parcial	`--only E-ent` seguia gastando GPU após FAIL	`TestContext.StopOnFail` + guard de rebuild (sha512) obrigatório com stop-on-fail	`U-circuit-battery-stop/resume`	ausente (sem runner de cenário)

Classe	Sintoma medido	Mecanismo (OCB)	Prova	Estado no `AI.Manager.Bridge`
Portão de fluxo	teste “bridge” podia passar sem MainHarness/tools tail	`GET /api/test/last-system` registra o system final (`fsmini`, `tail_scope`, `harness_tails`) — **portão único ainda pendente** (P1 do OCB)	pendente (`PENDINGS.md` P1)	ausente

9.6.2 Três lições que viram regra de arquitetura

1. **Mecânica no Core, texto no dado.** Contagem de falha, BFS, parse e seleção de ramo são código; tudo o que o modelo **lê** é dado versionado. O caso `ToolRetryEscalation` (criado para o loop de `oldString` e revertido por duplicar `ToolsAntiRepeat`) é a prova de que a duplicação de mecanismo é bug de processo.
2. **Medir antes de ditar.** O teto de passos, a janela de args e a regra de F# só entraram depois de medição com evidência (log do turno, `calls=`, probe de compilação). Nenhum número foi escolhido por intuição.
3. **Variância cognitiva não é regressão.** PASS isolado ≠ estabilidade: o mesmo binário pode falhar por escolha do modelo. Por isso o aceite é **3 verdes consecutivos por nível** com o mesmo hash de manifesto e rebuild entre rodadas.

9.7 Indução como dados — seeders e baseline de harness

Decisão (ADR-17): toda dinâmica corretiva que hoje vive como literal de código no OCB (`MainHarness`, `HarnessTails`, `FsMini`, `ToolHelp`, `ToolCallRepair`, `BridgeNotice`, `NudgeNotice`/`TripNotice`, dicas do `FsSyntaxRepair`) e todo enunciado hardcoded em teste (`E2EBelts*`, `E2EMission*`, `E2EEnt*`, `E-Science*`, `E-Front*`) devem existir no AI.Manager **como dado versionado**, com procedência (`source_commit` + `sha256`) e semeadura idempotente.

O algoritmo continua em código (§5.3 do guia): contar falhas, medir aridade, despachar por código FS, aplicar orçamento. **O que o modelo lê é dado.**

9.7.1 Modelo de dados

Reusa o que já existe e acrescenta o corpus de indução:

Tabela / contrato	Papel	Chave natural
`harness_profiles` → `harness_profile_versions` → `harness_attachments`	perfil do produto; anexos `Nucleo`, `Behavior`, `ToolsUsage`, `FsMini` (`kind`, `role`, `language`, `content`, `char_count`)	`(profile, version)`
`harness_assets` (nova)	corpus bruto do doador: `kind`, `role`, `language`, `text`, `sha256`, `source_commit`, `source_path`	`(source_commit, sha256)`
`harness_notices` (nova)	avisos/nudges/trip/salvage: `kind`, `tag`, `text`, `sha256` (tag `BRIDGE(...)` e `[BRIDGE-BREAKER]` em um lugar só)	`(kind, tag)`
`tool_help` (nova)	help curado por tool + schema de referência (`tool`, `text`, `sha256`)	`tool`
`hint_catalog` (nova)	dicas de reparo: `fs_code`, `symptom` (regex), `hint`, `probe` (`ok`/`pending`), `source_commit`	`(fs_code, symptom)`
`test_scenarios` (existe)	enunciados como dado (prompt, tools, loop, assertions)	`(plan_id, key)`
`runner_profiles` (existe)	Q4 gate / Q3 espelho, `context`, `max_tokens`	`runner_id`
`BridgeBaseline` (existe)	envelope de manifesto + runners, `source_commit`	`(source_commit, manifest_hash)`
`BridgeHarnessBaseline` (novo em `Bridge.Contracts`)	envelope do corpus de indução para transporte ao executor (o executor **não lê banco**)	`(source_commit, harness_hash)`

9.7.2 Seeders (idempotentes, com hash conferido)

Localização (ADR-18): todos em `src/Rochas.AI.Manager.Infrastructure/Seeding/` (assets em `Seeding/Assets/`); `TestBenchSeeder` fica em `Infrastructure/TestBench/`. **Nenhum seeder em `Web/Data`** — host só chama `SeedAsync()` no boot. O executor de cenário é console (`Bridge.Runner`) e não lê banco.

Seeder	Onde	Entrada	Regra
`HarnessBaselineSeeder`	`Infrastructure/Seeding`	`Data/Seeds/harness-baseline.json`	cria anexos `Draft`; nunca sobrescreve versão `Approved`; hash divergente reprova com mensagem explícita
`ToolHelpSeeder`	`Infrastructure/Seeding`	seção `toolHelp` do baseline	`ToolHelp.For` do Bridge passa a ler daqui; literal vira fallback
`HintCatalogSeeder`	`Infrastructure/Seeding`	seção `hints` do baseline	dica só entra com `probe=ok` (compila em `fs-repair-probe`); sem probe, fica `pending` e não é servida
`NoticeCatalogSeeder`	`Infrastructure/Seeding`	seção `notices`	tag e prefixo em um lugar só; consumidores (bridge, runner, Web) chamam o catálogo

Seeder	Onde	Entrada	Regra
`TestBenchSeeder` (existe)	`Infrastructure/TestBench`	`Data/Seeds/war-baseline.json`	`(source_commit, manifest_hash)`; belts como `test_scenarios`; Q4 gate/Q3 espelho
`HarnessSeeder` / `HarnessScenarioSeeder` / `ScenarioTemplateSeeder` / `BridgeFeatureSeeder` / `WorkflowSeeder` (existem em `Web/Data`)	**migrar** para `Infrastructure/Seeding`	assets atuais	Fase 0.5; sem mudança de conteúdo no mesmo commit

Formato do baseline de indução (arquivo próprio para não contaminar o hash do manifesto):

```
{
  "schemaVersion": "1.0",
  "sourceCommit": "3d8bd58",
  "sourceRepository": "rocha83/OpenCodeBridge",
  "harness": {
    "nucleo":   { "text": "...", "sha256": "..." },
    "behavior": { "withTools": { "text": "...", "sha256": "..." },
                  "withoutTools": { "text": "...", "sha256": "..." } },
    "tools":    { "text": "...", "sha256": "..." },
    "fsMini":   { "text": "...", "sha256": "..." }
  },
  "toolHelp": [ { "tool": "write", "text": "...", "sha256": "..." } ],
  "notices":  [ { "kind": "breaker", "tag": "BRIDGE-BREAKER", "text": "...", "sha256": "..." } ],
  "hints":    [ { "fsCode": "FS0010", "symptom": "Unexpected symbol ':' in binding",
                  "hint": "...", "probe": "ok" } ]
}
```

9.7.3 Consumo em runtime

1. Host (`Bridge.Api` / `Runner` / `AI.Manager.Api`) carrega o baseline uma vez no boot (arquivo ou DB), valida hash e publica `BridgeOptions.HarnessTexts`.
2. `MainHarness` / `HarnessTails` / `FsSyntaxRepair` / `ToolHelp` / `ToolCallRepair` **consomem** o texto recebido; o literal do Core permanece apenas como fallback declarado (log `harness_source=fallback`).
3. Seletores de dados, não de código: `FsMini` só com `language=fs`; cauda de tools só com `hasTools`; `Behavior` escolhe variante com/sem tools; `NudgeNotice` / `TripNotice` com `{N}` interpolado no ponto único.
4. O **executor de cenário** recebe o baseline no manifesto (JSON) e não acessa banco — mantém a decisão de F1/F2 (executor sem lock de SQLite).

9.7.4 Gates da indução como dados

Gate	Critério	Consequência
G-I1 Paridade de texto	`sha256(texto semeado) == sha256(literal do OCB em source_commit)` para núcleo/tails/FsMini/help/notices	divergência é **erro** de seed, não warning
G-I2 Dica válida	toda `hint_catalog` com `probe=ok` (compila) e sem literal inválido copyável no texto	dica `pending` não é servida
G-I3 Enunciado sem receita	lint bloqueante: proibido `MapGet/MapPost`, `useEffect`, nome exato de DI, “não use edit”, “grave PRIMEIRO”	seed reprovado na importação
G-I4 Orçamento medido	`loop_json.steps` preenchido com medição (`calls=`) e margem; 0 = “não medido” e não entra no gate	cenário sem orçamento não promove release
G-I5 Drift	relatório R31 compara `source_commit` do seed com o HEAD do doador	drift > 0 aparece no painel; absorção é decisão humana
G-I6 Portão de fluxo	request em raia bridge exige `main_harness`, `harness_tails` e `tools_tail` (quando `tools`) no system final	teste sem fluxo completo é **refutado** por instrumentação (P1 do OCB promovido)

---

10. Bridge e proxy (contrato real)

10.1 Caminho legado do AI.Manager (transitório)

`Api/Controllers/ProxyController.cs`, rota `POST /v1/chat/completions`:

1. Exige `Authorization: Bearer <qualquer não vazio>` (não valida o token).
2. Escolhe o **primeiro** `ModelBinding` (ignora o `model` do request para o binding).
3. Guarda de contexto: `chars/4 > context_window * 0.95` → HTTP 413.
4. Resolve `Agent` por nome/modelo e injeta `temperature` no body.
5. Encaminha para `http://localhost:4125` (provider `llama`) ou `:4124` (demais), com Bearer fixo `guest@123`.
6. Streaming SSE repassado linha a linha; não-stream passa por `ReasoningSanitizer` e `ToolCallCoercer`.
7. Grava `HarnessRun` (result code, duração, tps estimado por `chars/4`) — com `ProfileVersionId=0` e `EvidenceId=0`.

`POST /api/tools/execute` (`[Authorize]`) despacha 9 handlers leaf no workspace (`BridgeToolExecutor`): shell, read, write, edit, grep, glob, fetch, git, subagent — com `WorkspaceGuard`, `ShellInspector` e `ProcessRunner` (timeout 300s default).

10.2 AI.Manager.Bridge (porte 2026-10-25)

Projetos `net10.0` e dependências reais:

Projeto	Papel	Referencia
`Rochas.AI.Manager.Bridge`	núcleo protocol-only: `ResponsesToChat`, `ChatToResponses`, `BridgeProxy`, `MainHarness`, `HarnessTails`, `ContentAntiRepeat`, `ToolsAntiRepeat`, `ToolArgsParser`, `ToolCoercion`, `ToolSanitizer`, `TextGrammar`, `ResponseBody`, `ToolLoopContext`, `QwenStandard`, `Sampling`, `ContextGuard`	`Bridge.Contracts`
`Rochas.AI.Manager.Bridge.Contracts`	DTOs de opções/manifesto e schema; sem ASP.NET	—
`Rochas.AI.Manager.Bridge.Client`	HTTP/SSE tipado para `/v1/responses` e `/v1/chat/completions`	`Contracts`
`Rochas.AI.Manager.Bridge.Hosting`	`BridgeHostOptions` (flags/env), `BridgeComponent`, bootstrap	`Contracts`
`Rochas.AI.Manager.Bridge.Api`	host REST+Swagger	`Bridge`, `Contracts`, `Hosting`
`Rochas.AI.Manager.Bridge.Runner`	host CLI `serve` / `manifest` (HttpListener)	`Bridge`, `Contracts`, `Hosting`

Endpoints (Api e Runner): `GET /api/status`, `GET /health`, `GET /api/contract`, `GET /api/contract/schema`, `POST /api/manifest/validate`, `POST /api/manifest/dry-run`, `POST /v1/responses`, `POST /v1/chat/completions`.

Pipeline: `ResponsesToChat.Convert` → estratégia de tools → anti-repeat (conteúdo/args, Levenshtein + assinatura canônica) → breaker soft/hard → upstream OpenAI-compatible → `ChatToResponses`; stream SSE no mesmo contrato. `BridgeOptions.ForClient(User-Agent)` isola a decisão por requisição (o host é singleton e não muta a instância compartilhada).

Harness: `MainHarness` (base) + `HarnessTails` (delta), com `TailScope` e discriminador por cliente (§9.5). O core é testável sem ASP.NET; `Bridge.Api` e `Runner` são cascas.

10.3 Integração e cutover

- Hoje o AI.Manager ainda usa o `ProxyController` legado; `Bridge.Api` / `Runner` são hosts standalone e `Bridge.Client` já fala o contrato novo.
- Próximo corte: consumir `Bridge.Client` no `AI.Manager.Api` (in-process ou HTTP) e desligar o proxy legado por flag.
- O `OpenCodeBridge` permanece doador congelado; features novas entram no `AI.Manager.Bridge`. O arquivamento exige paridade + 3 gates Q4 verdes.
- **Q4 `:4124` é o gate**; **Q3 `:4125` é espelho pós-green** e nunca substitui o gate.

10.4 Invariantes da v2

- A bridge é **protocol-only**: não interpreta dataset, experimento, treino ou perfil.
- Token de entrada validado (JWT ou API key registrada); credencial do upstream vinda de configuração.
- Binding resolvido por `model` do request (nome de agente → role → binding), com fallback explícito e logado.
- `max_tokens` do cliente respeitado no repasse; ausência → valor do binding.
- `HarnessRun` sempre com `profile_version_id`, `model_binding_id`, `request_hash`; `evidence_id` quando a sonda estiver ligada.
- Toda resposta não-stream com `tool_calls` coercíveis é registrada; toda falha de upstream gera run com `result_code` real.
- `/v1/responses` e `/v1/chat/completions` têm testes de contrato no `Bridge.Tests` e no host.
- O core não referencia ASP.NET/CLI/Swagger; hosts são substituíveis.

10.5 Matriz de paridade `OCB Core` → `AI.Manager.Bridge` (2026-10-29)

Comparação direta de arquivos entre `../OpenCodeBridge/Rochas.OpenCodeBridge.Core` (HEAD `3d8bd58`) e `src/Rochas.AI.Manager.Bridge`:

Artefato do OCB	Papel	Estado no Bridge	Ação
`MainHarness.cs`	base do caminho gerido	**portado** (144 vs 143 linhas)	manter; consumir texto do seed
`HarnessTails.cs`	caudas + FsMini + `hasTools`	**parcial** (57 vs 119 linhas): falta `DefaultBehaviorNoTools`, `FsMini`, `Compose(..., hasTools)`	portar com texto vindo do baseline (§9.7)
`ToolsAntiRepeat.cs`	breaker/anti-repeat	**parcial** (343 vs 434 linhas): `FailureMarkers` na forma morta, sem alvo-mais-frequente, sem 2 alvos	completar mecânica + testes
`AntiRepeat.cs` / `ContentAntiRepeat.cs`	repetição de conteúdo	portado	manter janela própria de args
`ToolArgsParser.cs` / `ToolCoercion.cs` / `TextGrammar.cs`	parsing/coerção	portado	manter
`ToolCallRepair.cs`	salvage (nudge + exemplos + help)	**ausente**	portar mecânica; texto no catálogo

Artefato do OCB	Papel	Estado no Bridge	Ação
`ToolHelp.cs`	help curado por tool (fonte única)	**ausente**	portar; popular por `ToolHelpSeeder`
`BridgeNotice.cs`	formato único do aviso	**ausente** (prefixo espalhado)	portar; tag no catálogo
`FsSyntaxRepair.cs`	reparo F# medido na fonte	**ausente**	portar (517 linhas; maior lacuna)
`FsMini` (dentro de `HarnessTails`)	mini tutorial F# (1 linha por código FS)	**ausente**	entra como anexo `FsMini` semeado
`StackPreflight.cs`	preflight do stack (bridge/upstream/modelo)	**ausente**	portar para o runner de cenário
`RoleProfiles.cs`	temperatura/thinking por role	**ausente** no Bridge (perfis no produto)	seed de `ModelBinding` / `Agent`
`Metrics.cs` / `BridgeMetrics`	contadores e reset	parcial (`/api/metrics`, sem `/api/diag/reset`)	expor reset por bateria
`Tools/*` (catalog, guard, inspector, runner, handlers)	execução de tools	**portado** (F1)	manter
`ToolCatalog.cs`	shapes chat/Responses	**portado**	manter
Flow (driver/circuito/executor)	execução de cenário	**parcial** (só `ManifestCommandResolver`)	Fase F3 do roadmap

Regra da matriz: linha "ausente/parcial" é backlog, não licença para reimplementar no consumidor. O porte é sempre `mecânica no Core + texto no baseline + pin em C#`.

10.6 Orçamento de passos, fail-fast e portão de fluxo

- **Orçamento de passos é dado do cenário** (`loop_json.steps`), medido (`calls=`) e nunca ajustado para "fazer passar". Teto de referência medido no OCB: 24 (`E-ent-redis`, faixa da suíte 16–28).
- **Fail-fast em toda rodada**: primeiro FAIL aborta; `--no-stop-on-fail` é medição consciente, não padrão.
- **Guard de rebuild** obrigatório com fail-fast: PASS só com binário do hash corrente (`sha512`).
- **Reset antes de restart**: `POST /api/diag/reset?probe=1&files=1&panel=1` zera métricas, sonda, arquivos, enunciado e painel; restart fica para binário/pool. Granularidade: uma vez por bateria (nunca por step).
- **Portão de fluxo (P1 do OCB promovido a gate)**: ao fim de qualquer execução em raia bridge, o registro do **request efetivamente enviado** precisa atestar `main_harness`, `harness_tails` (scope ≠ off) e `tools_tail` quando houver `tools`; ausência refuta o teste por instrumentação, com dump do que existe. O `GET /api/test/last-system` do OCB é a base; no AI.Manager o registro vive no run de cenário (`test_evidence`).
- **Nunca criar bridge paralela** em porta inventada para "bancada": usa-se a unit alvo (`:4124` gate, `:4125` espelho, `:4126` CPU forçada), reiniciada no início da bateria.

---

11. Workflow, aprovação e auditoria

- `TrainingWorkflowCard` é o único agregado workflow-based hoje; `Move` grava transição com `approver_id = 0` (a corrigir: usar usuário autenticado).
- `workflow_steps` define o grafo por `entity_hash_code`; `WorkFlowBasedService<T>` implementa `GetEntitySteps/CanAdvance/Advance/GetTransitions`, mas **não está no DI** e é ignorado pelos serviços — os controllers usam regras `if` locais. A v2 exige um único caminho: serviço workflow resolvido no DI, com validação de passo antes de qualquer `Move`.
- `ApprovalsController` (Admin) agrega pendências de harness, experimentos, artefatos, perfis de instrução e cards; deve passar a consumir serviços, não repositórios.
- Auditoria: toda mudança de estado de agregado workflow-based gera `WorkFlowTransition` (de/para, ação, autor, nota, timestamp). Relatórios R22–R24 dependem disso.

---

12. Treinamento

12.1 Baseline atual (honesto)

- `POST /api/trainingjobs/{id}/run` executa `ITrainingEngine.RunAsync` **na requisição**.
- Básico: MLP `double[]` (entrada → 16 ocultos ReLU → saída), MSE, Adam, 10 épocas, checkpoint em memória não persistido; sem dataset, 256 amostras sintéticas; `vram_used_mb` sintético.
- Avançado: `python3 sft_train.py --dataset ... --method ...` via `ProcessRunner`.
- `TorchSharp` 0.107.0 referenciado e não usado; nenhum Safetensors, tokenizer, atenção, LoRA ou retomada.

12.2 Alvo e gates

Gate	Critério	Consequência
G1	Backend renomeado para refletir o que é (ex.: `MlpTrainingBackend`) e exposto como baseline	Elimina ambiguidade de capacidade
G2	Prova TorchSharp: tensor na GPU, autograd, Adam, loss caindo em dataset sintético	Habilita `TorchSharpTrainingBackend` real

Gate	Critério	Consequência
G3	Loader Safetensors + tokenizer + forward textual de 1 modelo	Habilita fine-tuning textual
G4	LoRA com pesos-base congelados, save/restore/resume	Habilita experimento real
G5	Fila persistente + lease + cancelamento cooperativo	Habilita execução assíncrona confiável

Decisão pendente (ADR-07): manter o caminho Python como *adaptador externo opcional* (desligado por default, declarado como backend externo) ou removê-lo. Enquanto não decidido, o rótulo de capacidade deve dizer "experimental/externo".

12.3 Fila e worker (alvo)

- Tabela `training_jobs` já serve de fila: `status`, `started_at`, `completed_at`, `error_message`.
- Worker como `BackgroundService` no host: `claim` atômico (`UPDATE ... WHERE id = (SELECT ...) AND status='Queued'`), lease por heartbeat, recuperação de `Running` órfão no boot.
- Um job por GPU: semáforo global + lock por artefato; cancelamento por `CancellationToken` propagado ao backend.
- Sem `TrainingWorker` separado na Fase 2; extração quando houver mais de um host.

---

13. Relatórios e analytics

13.1 Baseline (o que existe)

`ReportQuery` / `ReportResult` / `IReportService` + `ReportService` (13 entidades mapeadas) + `GET /api/reports/preview|export` + página `/Reports` com iframe e export CSV/XLSX/PDF/HTML. Filtro efetivo: **apenas `Status`**; paginação em memória; nenhum join.

13.2 Alvo (v2)

Registry declarativo (`ReportDefinition`), sem SQL montado a partir de entrada do usuário:

```
public sealed record ReportDefinition(
    string Id,                // "training-jobs-por-status"
    string Title,
    string RootTable,         // "training_jobs"
    IReadOnlyList<JoinSpec> Joins,
    IReadOnlyList<DimensionSpec> Dimensions,
    IReadOnlyList<MeasureSpec> Measures,
    IReadOnlyList<FilterSpec> Filters,
    DateAxis DefaultDateAxis,  // coluna + semântica
    IReadOnlyList<DateAxis> AllowedDateAxes,
    IReadOnlyList<string> ExportFormats);
```

Semântica temporal (ADR-05)

- Intervalo **half-open** em UTC: `[from 00:00:00, to+1 dia)`.
- Normalização: `yyyy-MM-dd HH:mm:ss` (SQLite TEXT).
- Eixos permitidos por relatório (nunca um eixo implícito): `created_at` (default), `started_at`, `completed_at`, `updated_at`, `approved_at`.
- "De" vazio = sem limite inferior; "Até" vazio = sem limite superior; ambos vazios = sem recorte (assumido, não silencioso).
- Relatório com dois eventos (ex.: duração) declara `started_at` e `completed_at` separadamente e calcula delta.

Pipeline

1. Valida `ReportDefinition` + filtros contra whitelist.
2. Monta SQL com parâmetros (nunca concatenação de valor).
3. `COUNT(*)` + página (`LIMIT/OFFSET`) no banco, não em memória.
4. Projeta `Dictionary<string, object?>` (compatível com `ReportResult`) ou DTO tipado.
5. Exporta via `Rochas.ExcelToJsonParser` / `Rochas.PDFGenerator`; PDF com MIME correto.

Endpoints

```
GET /api/reports/catalog                → definições, dimensões e eixos temporais
GET /api/reports/{id}?from&to&...       → execução paginada
GET /api/reports/{id}/export?format=    → csv|xlsx|pdf
GET /api/reports/preview                → compatibilidade (mapeia para definição default)
```

Catálogo

As 30 definições propostas (R1–R30), com facetas por FK, eixos temporais e métricas, estão em `REPORT-PENDINGS.md`. A v2 trata o catálogo como backlog priorizado, não como promessa de entrega simultânea.

Regras de qualidade

- Nenhum filtro é ignorado em silêncio: filtro não suportado pelo relatório retorna 400 com motivo.
- Toda dimensão de FK mostra código e nome (join explícito), nunca só o id.
- Relatórios de integridade (R29) e inventário (R30) são obrigatórios antes de qualquer migração de schema.

---

14. Segurança

Item	Estado	Alvo
Chave JWT	hardcoded `ai-manager-jwt-secret-change-me` em Web e Api	configuração/env/secret store; rotação
Seed de usuários	`admin/admin`, `teste@e2e.local/Teste@123` sempre	somente Development; troca forçada; remover em Release
Proxy `/v1`	aceita qualquer Bearer; envia `guest@123` ao upstream	validação real + credencial configurável
Autorização	`[Authorize]` em reports/tools; Web Admin em Approvals; demais abertas	política por controller; papéis explícitos
CSRF	antiforgery nos POSTs Web	manter; documentar exceções
Superfície de tools	`WorkspaceGuard` + `ShellInspector` + timeout	allowlist por role; auditoria de comando
Segredos em log	não há mascaramento	nunca logar Authorization/body sensível

15. Observabilidade e operação

- Log: Serilog console, nível Information com override `Microsoft=Warning`.
- `/health`: status, timestamp, `DIAGNOSTICS_ENABLED` (memória, uptime); alvo: `/health/ready` com checagem de SQLite e da bridge.
- Métricas mínimas: jobs por status, runs por result_code, duração p95, tps médio, capturas/dia.
- Diagnóstico de banco: caminho do arquivo, tamanho, WAL, última migração.
- Operação local: `dotnet run` em Web (`:5297` conforme `launchSettings`) e Api (Swagger em Development); DB no diretório do assembly.

16. Testes e qualidade

Tipo	Baseline	Alvo
Unitário	AccuracyEvaluator, EntityHashCode, helpers, linter, workflow	Cobrir invariantes de agregado e máquinas de estado
Integração	SQLite em `SequentialCollection`	Repositórios, transações, seed idempotente
Contrato	`Bridge.Tests`: 144 PASS (protocolo, harness, anti-repeat, breaker, tools, manifesto)	Golden files de `/v1/responses` e `/v1/chat/completions` no host
Web/API	—	`WebApplicationFactory` para rotas e exportação
Relatórios	4 casos	Golden CSV/XLSX por definição; De/Até inclusivo/exclusivo
Regressão de harness	—	Preview byte-idêntico ao request (hash)

Estado em 2026-10-29 (medido nesta revisão): `dotnet build Rochas.AI.Manager.slnx -c Release` = **0 erros / 113 avisos** (NU1902/NU1903 de `System.IO.Packaging` / `System.Security.Cryptography.Pkcs` no projeto de testes; CS9113/CS8625/CS0219); `Bridge.Tests` = **144 PASS**; `AI.Manager.Tests` = **105 PASS + 1 SKIP**. O doador `OpenCodeBridge` reporta `units` = **139 PASS / 0 FAIL / SKIP=6** no commit `3d8bd58`.

Regra: bug corrigido = teste que falha antes e passa depois. Teste que depende de GPU é marcado e roda em job separado.

17. ADRs (decisões registradas)

- **ADR-01 — Persistência:** SQLite + `Rochas.DapperRepository` como baseline. EF Core fora. Revisitar em PostgreSQL.
- **ADR-02 — FKs lógicas:** sem constraint física no SQLite; integridade por validação de escrita + relatório de órfãos.
- **ADR-03 — Harness versionado:** perfil/versão/anexos imutáveis após aprovação; bridge protocol-only; infraestrutura fora do prompt.
- **ADR-04 — Relatórios declarativos:** registry em código + SQL whitelisted; proibido SQL dinâmico a partir de query string.
- **ADR-05 — Intervalo temporal:** half-open em UTC, "Até" inclusivo no dia.
- **ADR-06 — Motor numérico:** MLP é baseline rotulado; TorchSharp só após prova (G2).
- **ADR-07 — Python:** adaptador externo opcional e desligado, ou remoção. Decisão do dono do produto.
- **ADR-08 — Chat:** portar `/Chat/{Stream,Sessions,Config,Approve,Calibrate,Stop,ClientLog}` para o AI.Manager ou remover a view; proibido segundo chat.
- **ADR-09 — Segurança:** segredos só em configuração; seed apenas em Development.
- **ADR-10 — Worker:** `BackgroundService` no host na Fase 2; projeto dedicado só com múltiplos hosts.
- **ADR-11 — Contratos de domínio:** `IRepository` / `IUnitOfWork` implementados ou removidos (sem código morto).
- **ADR-12 — Schema:** scripts versionados + `schema_version`; proibido `ALTER TABLE` em `try/catch`.
- **ADR-13 — Bridge extraível:** núcleo `Rochas.AI.Manager.Bridge` sem ASP.NET/CLI/Swagger; `Api` / `Runner` são hosts; `Contracts` / `Client` / `Hosting` completam o pacote. OpenCodeBridge é doador transitório; Q4 é gate e Q3 é espelho.
- **ADR-14 — Harness no Bridge:** `MainHarness` base + `HarnessTails` coexistentes; gate por cliente (`User-Agent`), nunca por sufixo de modelo; `TailScope` managed/all/off; `ForClient` isola a decisão por requisição.

- **ADR-15 — Indução é corpus medido:** toda dica/aviso/enunciado nasce de sintoma medido (log, `calls=`, probe de compilação) e obedece "a dica ensina a forma CERTA e nunca exibe a errada". Dica sem `probe=ok` não é servida.
- **ADR-16 — Breaker enxerga falha de tool:** `FailureMarkers` cobre `exit [1-9]`, `error fs/cs/msb/fsi\d+`, traceback e `undefined reference`; a contagem é por **alvo mais frequente na janela** (sucesso intercalado não zera), com nudge em até 2 alvos (soft 3 / hard 6). Consumidor de `/api/tools/execute` chama o mesmo controle — não cria escalada paralela.
- **ADR-17 — Indução como dados:** núcleo, tails (com/sem tools), `FsMini`, help por tool, notices e dicas vivem em baseline versionado (`source_commit` + `sha256`), semeados em idempotência; o Core consome e o literal é apenas fallback declarado. Hash divergente reprova.
- **ADR-18 — Seeder é Infrastructure; executor é console:** todo seeder e asset mora em `Infrastructure/Seeding` (banco é dela); `Web`/`Api` só chamam `SeedAsync()` no boot; o runner de cenário é `Bridge.Runner` (console), sem banco. Seeders em `Web/Data` são dívida a migrar (Fase 0.5).
- **ADR-19 — Diagnóstico por reset:** contadores, sonda, arquivos e painel zeram por `POST /api/diag/reset` (`?probe=1&files=1&panel=1`), uma vez por bateria; `step_start` não zera. Restart só para binário/pool.
- **ADR-20 — Orçamento e fail-fast medidos:** teto de passos é dado do cenário, medido com `calls=` e margem (referência 24); primeiro FAIL aborta a rodada; guard de rebuild (`sha512`) é obrigatório com stop-on-fail; aceite exige 3 gates verdes consecutivos por nível.
- **ADR-21 — Portão de fluxo:** execução em raia bridge só é válida se o request efetivamente enviado atesta `main_harness`, `harness_tails` (scope ≠ off) e `tools_tail` quando há `tools`; ausência refuta o teste por instrumentação (P1 do OCB promovido a gate).

---

18. Riscos

Risco	Prob.	Impacto	Sinal de alerta	Resposta
Declarar treino funcional sem prova	Alta	Alto	Job "Completed" sem loss/gradiente verificável	Gate G1/G2; rótulo honesto
Relatório entregar número errado	Alta	Alto	Filtro De/Até ausente; total ≠ filtrado	ADR-04/05 + testes golden
Chat quebrado em produção	Média	Médio	404 em `/Chat/Stream`	ADR-08
Vazamento de credencial	Média	Alto	Chave fixa em repo	ADR-09
Perda de evidência de experimento	Média	Alto	Run sem perfil/captura	§9.4
Crescimento do banco sem índice	Média	Médio	Consultas lentas em `harness_runs`	§8.2
Divergência harness nativo × custom	Média	Médio	Resultado muda sem mudança de versão	Diff + Test Bench
Duplicação proxy legado × Bridge	Alta	Médio	Dois caminhos com comportamento diferente	Cutover por flag + testes de contrato; remover o legado
Deriva OCB → AI.Manager.Bridge	Média	Alto	Correção só no doador	Sync com diff + gate Q4; ADR-13
Texto de indução em dois lugares (código e seed)	Alta	Alto	Comportamento muda sem mudar hash	ADR-17: paridade por `sha256` + fallback declarado
Seeder em camada errada (Web)	Média	Médio	Boot da Web criando catálogo; API sem dados	ADR-18: migrar para `Infrastructure/Seeding` + teste de boot
Dica inválida servida ao modelo	Média	Alto	Mesmo erro repetido em loop	ADR-15: `probe=ok` obrigatório e pin de ausência do literal errado
Breaker cego a falha de tool/compilador	Alta	Alto	Loop de GPU sem nudge/trip	ADR-16: marcadores + alvo-mais-frequente + teste de bateria
Orçamento de passos irreal	Média	Médio	FAIL de mecânica lido como regressão de modelo	ADR-20: teto medido (`calls=`) e `steps` como dado

---

19. Roadmap v2

Fase 0 — Verdade e segurança (1 sprint)
Renomear backend para o que ele é; decidir Python (ADR-07); chave JWT e seed configuráveis; menu com Reports/Approvals; corrigir MIME do PDF; corrigir `approver_id` nas transições.
Aceite: build verde; nenhum segredo no código; página de relatórios acessível pelo menu.

Fase 0.5 — Seeders na Infrastructure (2–3 dias)
Mover os 5 seeders de `Web/Data` (+ assets `harness-*.md` / `harness-tools.json`) para `Infrastructure/Seeding`; expor `SeedAsync()` na Infrastructure e chamá-lo no boot de Web/Api; nenhuma mudança de conteúdo no mesmo commit.
Aceite: `grep -rn "Seeder" src/Rochas.AI.Manager.Web` vazio; seed roda igual (teste de boot idempotente); Web/Api só chamam `SeedAsync()`.

Fase 1 — Relatórios confiáveis (1–2 sprints)
`ReportDefinition`, registry, filtros De/Até, facetas de FK, paginação em SQL, catálogo R1–R13 + R29/R30; export com MIME correto; testes golden.
Aceite: `from/to` altera o total; nenhum filtro ignorado; R29 acusa zero órfãos no seed.

Fase 2 — Execução assíncrona (1–2 sprints)
Fila em `training_jobs`, claim atômico, hosted service, cancelamento cooperativo, recuperação no boot, endpoint de status.
Aceite: `POST /run` retorna 202 imediato; job sobrevive a restart; cancelar interrompe em < 5s.

Fase 3 — Evidência de harness (1–2 sprints)
`HarnessRun` com perfil/versão/captura; Test Bench; diff de versões; matriz de capacidade.
Aceite: todo run rastreia perfil→versão→request→resultado; preview byte-idêntico.

Fase 3.5 — Bridge e cutover (em andamento)
`AI.Manager.Bridge` extraído (Core/Contracts/Client/Hosting/Api/Runner), `/v1/responses`, harness base+tails e anti-repeat/breaker. Falta consumir `Bridge.Client` no AI.Manager, desligar o proxy legado e congelar o doador.
Aceite: request do AI.Manager atravessa `Bridge.Client` → `Bridge.Api` com contrato e evidência; Q4 verde.

Fase 3.6 — Indução como dados (1–2 sprints, prioridade da rev. 2.2)
Porte do corpus de indução do OCB (`3d8bd58`) para o `AI.Manager.Bridge`, com texto vindo do baseline:
1. `harness-baseline.json` (`nucleo`, `behavior` com/sem tools, `tools`, `fsMini`, `toolHelp`, `notices`, `hints`) + `HarnessBaselineSeeder` / `ToolHelpSeeder` / `HintCatalogSeeder` / `NoticeCatalogSeeder` em `Infrastructure/Seeding`.
2. Mecânica: `HarnessTails.Compose(..., hasTools)`, `FsSyntaxRepair` (ramo por código FS medido na fonte + escalada de dica repetida), `ToolCallRepair` (nudge + 3 exemplos + help), `BridgeNotice` (tag única), `FailureMarkers` estendidos + alvo-mais-frequente, janela de args 1024, `WorkspaceGuard.Locate`, `StackPreflight`.
3. `POST /api/diag/reset` (`probe` / `files` / `panel`) + política de granularidade por bateria.
4. Portão de fluxo (ADR-21) no run de cenário, com `test_evidence` atestando harness/tails/tools.
5. Lint de enunciado no seed (G-I3) e relatório R31 de drift contra o HEAD do doador.
Aceite: paridade `sha256` de todos os textos com `3d8bd58` (G-I1); `hint_catalog` com `probe=ok` (G-I2); 1 cenário F# (`mission-algo-fs`) e 1 de tools (`tools-full`) verdes no Q4 com evidência; `Bridge.Tests` com os pins novos; nenhum literal de indução duplicado (`grep` gate).

Fase 4 — Motor numérico (2–4 sprints)
G2 (TorchSharp na GPU), loader Safetensors, forward textual de um modelo, LoRA, resume.
Aceite: loss cai em dataset sintético na GPU com gradientes finitos; adaptador salvo/restaurado.

Fase 5 — Operação e catálogo (1–2 sprints)
Dashboard operacional, estatísticas agregadas, `/health/ready`, índices, scripts de migração, integridade referencial contínua.
Aceite: painel mostra jobs/runs/erros com origem e timestamp; migração idempotente.

Fase 6 — Horizonte
Integração do `Bridge.Client` no AI.Manager (in-process/HTTP), cutover e remoção do proxy legado, clientes de inferência (Ollama/llama-server), multimídia/áudio, app mobile, PostgreSQL.

---

20. Critérios de aceitação da v2

1. Nenhuma capacidade anunciada sem teste e evidência (v1 §28.14).
2. Todo relatório respeita `from/to` e retorna 400 para filtro não suportado.
3. Job de treino nunca roda dentro da requisição HTTP.
4. Todo `HarnessRun` tem perfil, versão, binding e hash de request.
5. Nenhum segredo, credencial ou caminho local no código ou no prompt.
6. Bridge permanece protocol-only, com testes de contrato verdes.
7. Chat tem endpoints no AI.Manager ou deixa de existir como view.
8. Schema evolui por script versionado, com `schema_version`.
9. Integridade referencial verificável por relatório (R29) a qualquer momento.
10. Build e suíte verdes no CI antes de qualquer release.
11. `AI.Manager.Bridge` core não referencia ASP.NET/CLI/Swagger; `Api` / `Runner` são hosts substituíveis.
12. Harness no Bridge é decidido pelo cliente (`User-Agent`), com base+tails e `TailScope`; Q4 é gate e Q3 é espelho.
13. Todo texto de indução (núcleo, tails, FsMini, help, notices, dicas) é **dado versionado** com `sha256` e `source_commit`; literal no código é só fallback declarado.
14. Nenhum seeder mora em `Web` / `Api`; seed é `Infrastructure/Seeding` e o runner de cenário é console sem acesso a banco.
15. Todo enunciado semeado passa no lint G-I3 (sem mecânica ditada) e todo cenário declara orçamento de passos medido.
16. Execução em raia bridge sem `main_harness` / `harness_tails` / `tools_tail` no request é **refutada** por instrumentação, não por assertiva semântica.

---

21. Anexos

21.1 Mapa de arquivos-chave

Área	Arquivo
Schema	`src/Rochas.AI.Manager.Infrastructure/Data/AppDb.cs`
Repositórios/DI	`src/*/Program.cs`
Contratos de relatório	`src/Rochas.AI.Manager.Domain/Interfaces/IReportService.cs`
Execução de relatório	`src/Rochas.AI.Manager.Infrastructure/Services/ReportService.cs`
Proxy legado	`src/Rochas.AI.Manager.Api/Controllers/ProxyController.cs`
Bridge novo	`src/Rochas.AI.Manager.Bridge/*` + `Bridge.Api` / `Bridge.Runner`
Harness no Bridge	`src/Rochas.AI.Manager.Bridge/{MainHarness,HarnessTails}.cs`
Tools	`src/Rochas.AI.Manager.Application/Tools/*`
Helpers de resposta	`src/Rochas.AI.Manager.Application/Helpers/{ReasoningSanitizer,ToolCallCoercer}.cs`
Harness	`src/Rochas.AI.Manager.Domain/Harness/*` + `Application/Services/Harness*`
Workflow	`src/Rochas.AI.Manager.Domain/Entities/Management/*` + `Application/Services/{TrainingWorkflowService,WorkFlowBasedService}.cs`
Treino	`src/Rochas.AI.Manager.TrainingEngine/*`
Seed	`src/Rochas.AI.Manager.Infrastructure/Seeding/*` (alvo; hoje 5 seeders ainda em `Web/Data`)
Baseline de indução	`Data/Seeds/harness-baseline.json` + `Bridge.Contracts/BridgeHarnessBaseline.cs`
Corpus de indução (referência do doador)	`../OpenCodeBridge/Rochas.OpenCodeBridge.Core/{HarnessTails,FsSyntaxRepair,ToolCallRepair,ToolHelp,BridgeNotice}.cs`
TestBench	`Data/Seeds/war-baseline.json` + `Infrastructure/TestBench/*` + `Application/TestBench/ManifestBuilder.cs`
UI	`src/Rochas.AI.Manager.Web/Views/**` + `Controllers/**`

21.2 Glossário

- **Harness:** conjunto versionado de instruções, tails e políticas que molda o comportamento do agente.
- **Binding:** combinação provider + modelo + limites (contexto, max_tokens, thinking, temperatura).
- **Run:** execução observada de um harness contra um binding, com resultado e métricas.
- **Captura:** request/response completos coletados pela sonda.
- **Card:** unidade de aprovação no fluxo de treino.
- **Transição:** registro imutável de mudança de estado de entidade workflow-based.
- **Faceta:** dimensão de agrupamento/filtro derivada de coluna ou FK.
- **Eixo temporal:** coluna de data usada no recorte De/Até de um relatório.
- **MainHarness:** base enxuta que substitui o system do cliente no caminho gerido.
- **TailScope:** escopo das caudas (`managed` / `all` / `off`).
- **Gate Q4 / espelho Q3:** instância estável de aceite (`:4124`) e espelho pós-green (`:4125`).
- **Doador:** `OpenCodeBridge` congelado, consumido via `Bridge.Client` até o cutover.
- **Indução:** tudo o que o modelo lê (núcleo, tails, FsMini, help, avisos, dicas) e que molda o comportamento sem ser código.
- **Baseline de indução:** arquivo versionado com o corpus de indução + `source_commit` + `sha256` por item.
- **FsMini:** mini tutorial de F# injetado só na fase F# (uma linha por código `FSxxxx`, sintoma em prosa + forma correta).
- **FsSyntaxRepair:** reparo que mede o erro do compilador na fonte e devolve `ERRADO → CERTO` (ou silêncio).
- **Salvage:** resgate de tool call com args inválidos — remove a assistant quebrada e devolve um aviso com exemplos válidos.
- **Nudge / Trip:** aviso após N falhas no mesmo alvo (soft 3) e encerramento do turno sem chamar o modelo (hard 6).
- **Budget (orçamento de passos):** teto medido de tool calls por cenário (`loop.steps`), com `calls=` na evidência.
- **Portão de fluxo:** verificação de que o request em raia bridge realmente carregou `main_harness` + tails (+ tools tail).
- **Drift:** distância entre o `source_commit` do seed e o HEAD do doador; relatório R31.

21.3 Change log

Versão	Data	Mudança
v1.2	2026-10-19	Guideline original + seção 31 (harness)

Versão	Data	Mudança
v1.3	2026-10-20	Seção 32 (as-built) e nota de revisão
v2.0	2026-10-20	Consolidação do as-built, subsistema de relatórios, ADRs, roadmap e critérios de aceitação
v2.1	2026-10-25	Porte do `AI.Manager.Bridge` (Core/Contracts/Client/Hosting/Api/Runner), harness base+tails, `TailScope`, discriminador por cliente; C4 regenerado
v2.2	2026-10-29	Panorama da indução do OCB (127 commits), matriz de paridade OCB Core → Bridge (§10.5), indução como dados e seeders (§9.7), seeders na Infrastructure e runner console (ADR-18), ADR-15…ADR-21, Fase 0.5 e Fase 3.6, critérios 13–16; PDF regenerado